Celery + Redis Broker Design — b-secure Platform

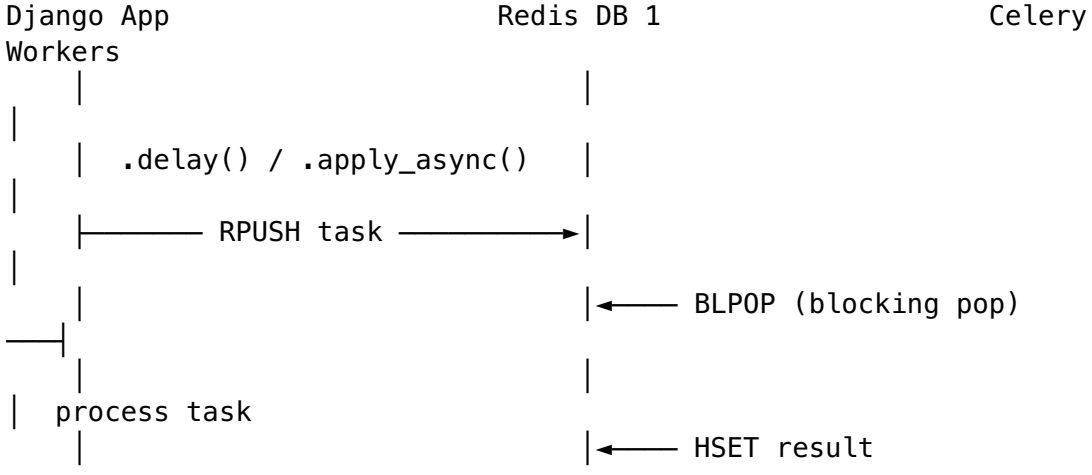
> Production Celery design reference: queue architecture, Redis key patterns, retry strategies, beat schedule, and deployment.

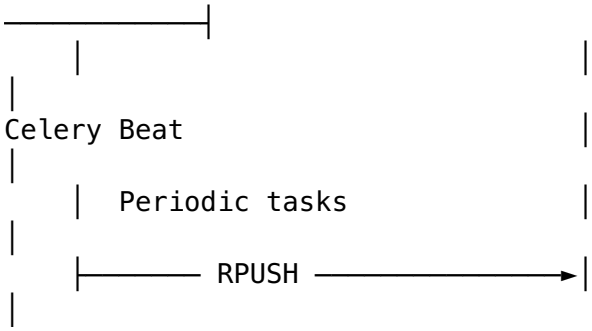
Table of Contents

---

1. Overview

Celery handles all asynchronous and scheduled work in b-secure. Redis (DB 1) acts as both the **message broker** and the **result backend**.





Why Redis over RabbitMQ for this project: - Simpler ops — one fewer managed service on AWS - ElastiCache already provisioned for cache - Task volumes are moderate (< 1000 tasks/min peak) - Result backend on the same Redis instance reduces round trips

Trade-off: Redis broker does not support task acknowledgement guarantees as robustly as RabbitMQ. Mitigated with `acks_late=True` and `task_reject_on_worker_lost=True` on critical tasks.

---

2. Celery App Configuration

```python
# celery_app.py (project root, loaded by Django)
import os
from celery import Celery
from celery.schedules import crontab

os.environ.setdefault("DJANGO_SETTINGS_MODULE",
        "config.settings.production")

app = Celery("bsecure")
app.config_from_object("django.conf:settings", namespace="CELERY")
app.autodiscover_tasks()

# settings/base.py — all Celery settings prefixed with CELERY_
from celery.schedules import crontab

# Broker and backend
CELERY_BROKER_URL       = env("REDIS_CELERY_URL",
        default="redis://localhost:6379/1")
CELERY_RESULT_BACKEND   = env("REDIS_CELERY_URL",
        default="redis://localhost:6379/1")

# Serialisation
CELERY_TASK_SERIALIZER    = "json"
CELERY_RESULT_SERIALIZER  = "json"
CELERY_ACCEPT_CONTENT     = ["json"]

# Timezone
CELERY_TIMEZONE           = "Asia/Kolkata"
```

```
CELERY_ENABLE_UTC         = True         # store internally as
        UTC, display in IST

# Task behaviour
CELERY_TASK_TRACK_STARTED            = True    # task enters
        STARTED state before running
CELERY_TASK_ACKS_LATE                = True    # ack AFTER task
        completes (safer)
CELERY_TASK_REJECT_ON_WORKER_LOST    = True    # re-queue if
        worker dies mid-task
CELERY_WORKER_PREFETCH_MULTIPLIER    = 1       # prevent a single
        worker hoarding tasks
CELERY_TASK_ALWAYS_EAGER             = False
        # set True in tests only

# Results
CELERY_RESULT_EXPIRES     = 3600
        # task results expire after 1 hour
CELERY_TASK_IGNORE_RESULT = False
        # store results (needed for retries + Flower)

# Rate limits
CELERY_TASK_DEFAULT_RATE_LIMIT = "100/m"

# Worker
CELERY_WORKER_MAX_TASKS_PER_CHILD   = 500    # restart worker
        after 500 tasks (memory leaks)
CELERY_WORKER_MAX_MEMORY_PER_CHILD  = 200000
        # 200 MB — restart if exceeded
```

---

3. Queue Architecture

3.1 Queue Definitions

Queue	Priority	Concurrency	Purpose
`high_priority`	Highest	4	Booking acceptance, SOS alerts, payment confirmation
`default`	Normal	8	Notifications, OTP SMS, status updates
`low_priority`	Low	4	Email reports, analytics, non-urgent cleanup
`scheduled`	Periodic	2	

Queue	Priority	Concurrency	Purpose
			Beat-triggered periodic tasks only

3.2 Task → Queue Routing Table

Task Name	Queue	Rationale
`send_booking_request_to_guard`	`high_priority`	Guard must receive within seconds
`process_booking_payment`	`high_priority`	Payment must not be delayed
`trigger_sos_alert`	`high_priority`	Safety-critical
`send_booking_confirmation_notification`	`default`	User notification (2–5s acceptable)
`send_otp_sms`	`default`	OTP must arrive within 30s
`update_guard_average_rating`	`default`	After review submission
`send_booking_receipt_email`	`low_priority`	Receipt can be slightly delayed
`generate_guard_earnings_report`	`low_priority`	Background report
`cleanup_expired_otps`	`scheduled`	Periodic cleanup
`process_pending_payouts`	`scheduled`	Nightly payout run
`generate_daily_analytics`	`scheduled`	Analytics aggregation
`refresh_guard_leaderboard`	`scheduled`	Leaderboard refresh

3.3 task_routes Configuration

```
# settings/base.py
CELERY_TASK_ROUTES = {
    # High priority
    "bookings.tasks.send_booking_request_to_guard":
        {"queue": "high_priority"},
    "payments.tasks.process_booking_payment":
        {"queue": "high_priority"},
    "sos.tasks.trigger_sos_alert":
        {"queue": "high_priority"},
    "sos.tasks.escalate_sos_event":
        {"queue": "high_priority"},
    "bookings.tasks.accept_booking_notification":
        {"queue": "high_priority"},

    # Default
    "notifications.tasks.send_push_notification":
        {"queue": "default"},
    "notifications.tasks.send_otp_sms":
        {"queue": "default"},
    "notifications.tasks.send_booking_confirmation":
        {"queue": "default"},
    "reviews.tasks.update_guard_average_rating":
        {"queue": "default"},
    "bookings.tasks.send_booking_status_update":
        {"queue": "default"},
    "guards.tasks.process_document_verification":
        {"queue": "default"},

    # Low priority
    "notifications.tasks.send_booking_receipt_email":
        {"queue": "low_priority"},
    "reports.tasks.generate_guard_earnings_report":
        {"queue": "low_priority"},
    "exports.tasks.generate_admin_export":
        {"queue": "low_priority"},
    "analytics.tasks.compute_guard_utilization":
        {"queue": "low_priority"},

    # Scheduled (beat only)
    "tasks.cleanup_expired_otps":
        {"queue": "scheduled"},
    "tasks.process_pending_payouts":
        {"queue": "scheduled"},
    "tasks.generate_daily_analytics":
        {"queue": "scheduled"},
    "tasks.refresh_guard_leaderboard":
        {"queue": "scheduled"},
    "tasks.send_weekly_guard_earnings_report":
        {"queue": "scheduled"},
    "tasks.cleanup_old_location_snapshots":
        {"queue": "scheduled"},
```

```
    "tasks.health_check_ping":
        {"queue": "scheduled"},
}
```

3.4 Worker Startup Commands

```
# High priority worker — 4 concurrent, dedicated
celery -A celery_app worker \
    --queues=high_priority \
    --concurrency=4 \
    --hostname=worker-high@%h \
    --loglevel=info \
    --max-tasks-per-child=500

# Default worker — 8 concurrent (handles most volume)
celery -A celery_app worker \
    --queues=default \
    --concurrency=8 \
    --hostname=worker-default@%h \
    --loglevel=info \
    --max-tasks-per-child=500

# Low priority worker — 4 concurrent
celery -A celery_app worker \
    --queues=low_priority \
    --concurrency=4 \
    --hostname=worker-low@%h \
    --loglevel=info \
    --max-tasks-per-child=500

# Scheduled / beat worker — 2 concurrent, single instance
celery -A celery_app worker \
    --queues=scheduled \
    --concurrency=2 \
    --hostname=worker-beat@%h \
    --loglevel=info

# Beat scheduler — single instance only, never run multiple beats
celery -A celery_app beat \
    --scheduler=django_celery_beat.schedulers:DatabaseScheduler \
    --loglevel=info
```

---

4. Redis Key Patterns for Celery

Celery uses the `kombu` library to interact with Redis. Understanding the key patterns is essential for monitoring and debugging.

4.1 Broker Queue Keys

```
_kombu.binding.high_priority    — Redis List (tasks stored as JSON
strings)
_kombu.binding.default
_kombu.binding.low_priority
_kombu.binding.scheduled
```

Tasks are **enqueued with RPUSH** (append to right) and **consumed with BLPOP** (blocking pop from left) — FIFO order.

```
# Check queue depth
redis-cli -n 1 LLEN _kombu.binding.high_priority
redis-cli -n 1 LLEN _kombu.binding.default

# Peek at the next task in queue (without consuming it)
redis-cli -n 1 LINDEX _kombu.binding.default 0

# View all tasks in a queue (careful with large queues)
redis-cli -n 1 LRANGE _kombu.binding.default 0 9
```

4.2 Task Result Keys

```
celery-task-meta-{task_uuid}    — Redis String (JSON), TTL =
result_expires (3600s)

# Check a specific task result
redis-cli -n 1 GET celery-task-meta-550e8400-e29b-41d4-
        a716-446655440000

# Count pending results
redis-cli -n 1 --scan --pattern 'celery-task-meta-*' | wc -l
```

4.3 Worker Heartbeat Keys

```
_kombu.{worker_hostname}.heartbeat     — Worker liveness key
celery.worker.revoked                  — Set of revoked task IDs

# List all registered workers
redis-cli -n 1 KEYS '_kombu.*.heartbeat'

# Revoke a specific task
celery -A celery_app control revoke {task_id} --terminate
```

---

5. Task Payload Structure

5.1 Full Celery Task JSON Payload


```
{
  "body":
        "eyJpZCI6ICI1NTBlODQwMC0uLi4iLCAidGFzayI6ICJub3RpZmljYXRpb25zLnRhc2tzLnNlb
  "content-encoding": "utf-8",
  "content-type": "application/json",
  "headers": {
    "lang": "py",
    "task": "notifications.tasks.send_booking_confirmation",
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "shadow": null,
    "eta": null,
    "expires": null,
    "group": null,
    "group_index": null,
    "retries": 0,
    "timelimit": [null, null],
    "root_id": "550e8400-e29b-41d4-a716-446655440000",
    "parent_id": null,
    "argsrepr": "()",
    "kwargsrepr": "{'booking_id': 4891}",
    "origin": "gen12345@worker-default"
  },
  "properties": {
    "correlation_id": "550e8400-e29b-41d4-a716-446655440000",
    "reply_to": "a1b2c3d4-results",
    "delivery_mode": 2,
    "delivery_info": {
      "exchange": "",
      "routing_key": "default"
    },
    "priority": 0,
    "body_encoding": "base64",
    "delivery_tag": "a1b2c3d4-e5f6-..."
  }
}
```


5.2 Example: send_booking_confirmation_notification

```python
# notifications/tasks.py
from celery import shared_task
from django.conf import settings
import logging

logger = logging.getLogger(__name__)


@shared_task(
```

```
    name="notifications.tasks.send_booking_confirmation",
    bind=True,
    max_retries=3,
    default_retry_delay=60,
    acks_late=True,
    queue="default",
)
def send_booking_confirmation_notification(self, booking_id: int):
    """
    Send push notification + SMS to user and guard confirming
        booking creation.
    Task payload kwargs: {"booking_id": 4891}
    """
    from bookings.models import Booking
    from notifications.service import push_service, sms_service

    try:
        booking = Booking.objects.select_related(
            "user", "guard"
        ).get(id=booking_id)
    except Booking.DoesNotExist:
        logger.error("send_booking_confirmation: booking %s not
        found", booking_id)
        return  # no retry — data issue

    try:
        push_service.send_to_user(
            user_id=booking.user_id,
            title="Booking Confirmed",
            body=f"Your guard {booking.guard.full_name} will
        arrive at {booking.scheduled_at:%I:%M %p}.",
            data={"booking_id": booking_id, "type":
        "booking_confirmed"},
        )
        push_service.send_to_guard(
            guard_id=booking.guard_id,
            title="New Booking",
            body=f"Booking #{booking_id} confirmed for
        {booking.scheduled_at:%d %b %I:%M %p}.",
            data={"booking_id": booking_id, "type":
        "booking_confirmed"},
        )
    except Exception as exc:
        logger.warning("Push notification failed for booking %s:
        %s", booking_id, exc)
        raise self.retry(exc=exc, countdown=60)
```

5.3 Example: process_booking_payment

```
# payments/tasks.py
from celery import shared_task
import logging
```

```python
logger = logging.getLogger(__name__)


@shared_task(
    name="payments.tasks.process_booking_payment",
    bind=True,
    max_retries=5,
    acks_late=True,
    queue="high_priority",
    autoretry_for=(ConnectionError, TimeoutError),
    retry_backoff=True,         # exponential backoff
    retry_backoff_max=300,      # max 5 min between retries
    retry_jitter=True,
)
def process_booking_payment(self, booking_id: int, amount_paise:
        int, razorpay_order_id: str):
    """
    Capture payment and credit guard wallet.
    Task payload kwargs: {"booking_id": 4891, "amount_paise":
        150000, "razorpay_order_id": "order_xyz"}
    """
    from payments.service import PaymentService

    try:
        PaymentService.capture_and_settle(
            booking_id=booking_id,
            amount_paise=amount_paise,
            razorpay_order_id=razorpay_order_id,
        )
    except PaymentService.AlreadyProcessed:
        logger.info("Payment already processed for booking %s —
        skipping", booking_id)
        return
    except PaymentService.PaymentFailed as exc:
        logger.error("Payment failed for booking %s: %s",
        booking_id, exc)
        # Notify admin + update booking status
        from bookings.models import Booking
        Booking.objects.filter(id=booking_id).update(status="payment_failed")
        raise  # do not retry on business logic failure
```

---

6. Retry & Error Handling

6.1 Manual Retry with Exponential Backoff

```python
@shared_task(bind=True, max_retries=5)
def my_task(self, booking_id: int):
    try:
        do_work(booking_id)
    except (ConnectionError, TimeoutError) as exc:
```

```
        # Exponential backoff: 2, 4, 8, 16, 32 seconds
        countdown = 2 ** self.request.retries
        logger.warning(
            "Transient error on attempt %d, retrying in %ds: %s",
            self.request.retries + 1, countdown, exc
        )
        raise self.retry(exc=exc, countdown=countdown)
    except Exception as exc:
        # Permanent failure — log, alert, do not retry
        logger.exception("Permanent failure for booking %s",
        booking_id)
        sentry_sdk.capture_exception(exc)
        raise
```

6.2 autoretry_for — Automatic Retry on Specific Exceptions

```
@shared_task(
    bind=True,
    autoretry_for=(requests.exceptions.ConnectionError,
        requests.exceptions.Timeout),
    retry_kwargs={"max_retries": 3, "countdown": 30},
    retry_backoff=True,
    retry_jitter=True,
)
def send_sms_via_gateway(self, phone: str, message: str):
    response = requests.post(SMS_GATEWAY_URL, json={"to": phone,
        "body": message}, timeout=10)
    response.raise_for_status()
```

6.3 Task Timeout

```
# Hard timeout: worker process is killed after 300s
# Soft timeout: SoftTimeLimitExceeded exception raised at 270s
@shared_task(
    time_limit=300,        # hard limit (SIGKILL)
    soft_time_limit=270,   # soft limit (SoftTimeLimitExceeded —
        can clean up)
)
def generate_large_report(guard_id: int):
    from celery.exceptions import SoftTimeLimitExceeded
    try:
        build_report(guard_id)
    except SoftTimeLimitExceeded:
        logger.warning("Report generation timed out for guard
        %s", guard_id)
        cleanup()
```

6.4 On Permanent Failure

```
# celery_app.py — global failure signal handler
from celery.signals import task_failure
```

```
import sentry_sdk


@task_failure.connect
def handle_task_failure(sender, task_id, exception, args, kwargs,
        traceback, einfo, **kw):
    sentry_sdk.capture_exception(exception)
    # Optionally save to dead letter store (see section 8)
    save_to_dead_letter(
        task_name=sender.name,
        task_id=task_id,
        args=args,
        kwargs=kwargs,
        error=str(exception),
    )
```

---

7. Task Deduplication (Idempotency)

7.1 Redis SET NX Pattern

```
# core/task_dedup.py
from core.redis_client import get_redis
import functools
import logging

logger = logging.getLogger(__name__)


def idempotent_task(ttl_seconds: int = 3600, key_fn=None):
    """
    Decorator to prevent duplicate task execution within a time
        window.
    key_fn: callable that receives *args, **kwargs and returns the
        unique key suffix.
    Default: uses first positional arg (e.g., booking_id).
    """
    def decorator(task_func):
        @functools.wraps(task_func)
        def wrapper(*args, **kwargs):
            unique_part = key_fn(*args, **kwargs) if key_fn else
        str(args[0] if args else "default")
            dedup_key   = f"bsecure:celery:dedup:
        {task_func.__name__}:{unique_part}"
            r           = get_redis()

            # SET NX — only set if key does not exist
            acquired = r.set(dedup_key, "1", nx=True,
        ex=ttl_seconds)
            if not acquired:
                logger.info(
```

```
                    "Duplicate task skipped: %s [key=%s]",
                    task_func.__name__, dedup_key
                )
                return None   # silently skip

            return task_func(*args, **kwargs)
        return wrapper
    return decorator
```

7.2 Usage — Prevent Double-Charging

```
from core.task_dedup import idempotent_task


@shared_task(bind=True, queue="high_priority")
@idempotent_task(
    ttl_seconds=3600,
    key_fn=lambda booking_id, **kw: str(booking_id),
)
def charge_for_completed_booking(self, booking_id: int):
    """
    Guaranteed to run at most once per booking_id per hour.
    Even if called multiple times (webhook retries, manual retry),
        only one charge occurs.
    """
    from payments.service import PaymentService
    PaymentService.charge_completed(booking_id)
```

7.3 Celery Task Idempotency Checklist

- [ ] Payment tasks: deduplicate on `booking_id`
- [ ] Notification tasks: deduplicate on `{booking_id}:{notification_type}`
- [ ] OTP tasks: deduplicate on `{phone}:{otp_type}` with short TTL (60s)
- [ ] Report generation: deduplicate on `{guard_id}:{date}`
- [ ] Webhook handlers: check `external_reference` in DB before processing

---

8. Dead Letter Handling

Celery has no built-in dead-letter queue. Implement with a custom on_failure handler backed by PostgreSQL.

8.1 FailedTask Model

```
# core/models.py
from django.db import models


class FailedTask(models.Model):
    task_name   = models.CharField(max_length=255)
    task_id     = models.UUIDField()
    args        = models.JSONField(default=list)
    kwargs      = models.JSONField(default=dict)
    error       = models.TextField()
    traceback   = models.TextField(blank=True)
    retried_at  = models.DateTimeField(null=True, blank=True)
    resolved    = models.BooleanField(default=False)
    created_at  = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ["-created_at"]
        indexes  = [
            models.Index(fields=["task_name", "resolved"]),
            models.Index(fields=["created_at"]),
        ]

    def __str__(self):
        return f"{self.task_name} [{self.task_id}] —
        {self.error[:80]}"
```

8.2 Save to Dead Letter Store

```
# core/dead_letter.py
from core.models import FailedTask
import logging

logger = logging.getLogger(__name__)


def save_to_dead_letter(task_name: str, task_id: str, args,
        kwargs, error: str, traceback: str = ""):
    try:
        FailedTask.objects.create(
            task_name=task_name,
            task_id=task_id,
            args=list(args),
            kwargs=dict(kwargs),
            error=error,
            traceback=traceback,
        )
    except Exception as e:
        logger.critical("Failed to save task to dead letter
        store: %s", e)
```

8.3 Admin Command to Retry Failed Tasks

```
# management/commands/retry_failed_tasks.py
from django.core.management.base import BaseCommand
from core.models import FailedTask
from django.utils import timezone
import importlib


class Command(BaseCommand):
    help = "Retry tasks in the dead letter store"

    def add_arguments(self, parser):
        parser.add_argument("--task-name", type=str, default=None)
        parser.add_argument("--limit",     type=int, default=50)

    def handle(self, *args, **options):
        qs = FailedTask.objects.filter(resolved=False)
        if options["task_name"]:
            qs = qs.filter(task_name=options["task_name"])
        qs = qs[:options["limit"]]

        for failed in qs:
            try:
                module_path, func_name =
        failed.task_name.rsplit(".", 1)
                module = importlib.import_module(module_path)
                task   = getattr(module, func_name)
                task.apply_async(args=failed.args,
        kwargs=failed.kwargs)

                failed.retried_at = timezone.now()
                failed.resolved   = True
                failed.save(update_fields=["retried_at",
        "resolved"])

                self.stdout.write(self.style.SUCCESS(f"Retried:
        {failed.task_name} [{failed.task_id}]"))
            except Exception as e:
                self.stderr.write(f"Failed to retry
        {failed.task_id}: {e}")
```

---

9. Beat Schedule

9.1 Full beat_schedule

```
# settings/base.py
from celery.schedules import crontab

CELERY_BEAT_SCHEDULE = {
```

```
    # ── Every 1 minute ──────────────────────────────────────────
    "sync-guard-online-status": {
        "task":     "tasks.sync_guard_online_status",
        "schedule": 60.0,   # seconds
        "options":  {"queue": "scheduled", "expires": 55},
    },
    "health-check-ping": {
        "task":     "tasks.health_check_ping",
        "schedule": 60.0,
        "options":  {"queue": "scheduled", "expires": 55},
    },

    # ── Every 2 minutes ─────────────────────────────────────────
    "check-pending-bookings-timeout": {
        "task":     "tasks.check_pending_bookings_timeout",
        "schedule": 120.0,
        "options":  {"queue": "scheduled", "expires": 110},
        # Cancels bookings in 'pending' state for > 10 minutes
        with no guard acceptance
    },

    # ── Every 5 minutes ─────────────────────────────────────────
    "check-overdue-checkins": {
        "task":     "tasks.check_overdue_checkins",
        "schedule": 300.0,
        "options":  {"queue": "scheduled", "expires": 290},
    },
    "check-active-sos-escalation": {
        "task":     "tasks.check_active_sos_escalation",
        "schedule": 300.0,
        "options":  {"queue": "scheduled", "expires": 290},
        # Escalates SOS events not acknowledged within 3 minutes
    },

    # ── Every 15 minutes ────────────────────────────────────────
    "send-booking-reminders": {
        "task":     "tasks.send_booking_reminders",
        "schedule": 900.0,
        "options":  {"queue": "scheduled", "expires": 890},
        # Sends reminder 30 min before scheduled_at
    },

    # ── Every 30 minutes ────────────────────────────────────────
    "refresh-guard-leaderboard": {
        "task":     "tasks.refresh_guard_leaderboard",
        "schedule": 1800.0,
        "options":  {"queue": "scheduled", "expires": 1790},
    },
```

```
    # ── Every 1 hour ──────────────────────────────────────────────────────
    "cleanup-expired-otps": {
        "task":     "tasks.cleanup_expired_otps",
        "schedule": crontab(minute=0),   # top of every hour
        "options":  {"queue": "scheduled"},
    },

    # ── Daily at midnight IST (18:30 UTC) ─────────────────────────────────
    "generate-daily-analytics": {
        "task":     "tasks.generate_daily_analytics",
        "schedule": crontab(hour=18, minute=30),   # midnight IST
        = 18:30 UTC
        "options":  {"queue": "scheduled"},
    },

    # ── Daily at 2 AM IST (20:30 UTC previous day) ────────────────────────
    "process-pending-payouts": {
        "task":     "tasks.process_pending_payouts",
        "schedule": crontab(hour=20, minute=30),
        # 2:00 AM IST = 20:30 UTC
        "options":  {"queue": "scheduled"},
        # Processes guard earnings payouts via Razorpay Route
    },

    # ── Weekly: Monday 9 AM IST (Monday 03:30 UTC) ────────────────────────
    "send-weekly-guard-earnings-report": {
        "task":     "tasks.send_weekly_guard_earnings_report",
        "schedule": crontab(hour=3, minute=30,
        day_of_week="monday"),
        "options":  {"queue": "scheduled"},
    },

    # ── Weekly: Sunday 2 AM IST ───────────────────────────────────────────
    "cleanup-old-location-snapshots": {
        "task":     "tasks.cleanup_old_location_snapshots",
        "schedule": crontab(hour=20, minute=30,
        day_of_week="saturday"),
        # Saturday 20:30 UTC = Sunday 02:00 IST
        "options":  {"queue": "scheduled"},
        # Deletes tracking_locationsnapshot rows older than 90
        days
    },
}
```

9.2 IST ⍰ UTC Conversion Reference

IST Time	UTC Time	crontab()
Midnight (00:00)	18:30 (prev day)	`crontab(hour=18, minute=30)`
2:00 AM	20:30 (prev day)	`crontab(hour=20, minute=30)`
9:00 AM	3:30	`crontab(hour=3, minute=30)`
12:00 PM	6:30	`crontab(hour=6, minute=30)`
6:00 PM	12:30	`crontab(hour=12, minute=30)`

> **Important:** `CELERY_ENABLE_UTC=True` means beat interprets crontab times in UTC. Always convert IST to UTC for crontab schedules.

---

10. Monitoring

10.1 Flower Setup

```
# Install
pip install flower

# Run (with basic auth in production)
celery -A celery_app flower \
    --port=5555 \
    --broker=redis://:password@redis:6379/1 \
    --basic_auth=admin:supersecretpassword \
    --persistent=True \
    --db=/var/lib/flower/flower.db

# Or via Docker
docker run -p 5555:5555 mher/flower \
    celery flower \
    --broker=redis://:password@redis:6379/1
```

Useful Flower API endpoints:

Endpoint	Description
`GET /api/workers`	All workers + their stats
`GET /api/tasks?state=FAILURE&limit=20`	Recent failed tasks
`GET /api/queues/length`	Queue depths
`POST /api/task/apply/{task_name}`	Trigger a task manually
`POST /api/worker/shutdown/{worker}`	Graceful worker shutdown

10.2 Redis Queue Depth Monitoring Script

```
# scripts/monitor_queues.py
import redis
import sys

QUEUES = ["high_priority", "default", "low_priority", "scheduled"]
WARNING_THRESHOLD  = 100
CRITICAL_THRESHOLD = 500

r = redis.Redis(host="redis", port=6379, db=1,
        decode_responses=True)

exit_code = 0
for queue in QUEUES:
    depth = r.llen(f"_kombu.binding.{queue}")
    status = "OK"
    if depth >= CRITICAL_THRESHOLD:
        status    = "CRITICAL"
        exit_code = 2
    elif depth >= WARNING_THRESHOLD:
        status    = "WARNING"
        exit_code = max(exit_code, 1)

    print(f"{status:8s} | {queue:20s} | depth={depth}")

sys.exit(exit_code)
```

10.3 Celery Signals for Logging/Metrics

```
# celery_app.py
from celery.signals import task_prerun, task_postrun,
        task_failure, task_retry
import time
import logging

logger = logging.getLogger("celery.metrics")

_task_start_times = {}


@task_prerun.connect
def task_started(task_id, task, args, kwargs, **kw):
    _task_start_times[task_id] = time.monotonic()
    logger.info("TASK_START task=%s id=%s", task.name, task_id)


@task_postrun.connect
def task_finished(task_id, task, args, kwargs, retval, state,
        **kw):
    duration = time.monotonic() - _task_start_times.pop(task_id,
        time.monotonic())
```

```
    logger.info("TASK_END task=%s id=%s state=%s duration=%.3fs",
        task.name, task_id, state, duration)
    # Send to Prometheus/StatsD here


@task_failure.connect
def task_failed(task_id, exception, args, kwargs, traceback,
        sender, **kw):
    logger.error("TASK_FAIL task=%s id=%s error=%s", sender.name,
        task_id, exception)


@task_retry.connect
def task_retrying(request, reason, einfo, **kw):
    logger.warning("TASK_RETRY task=%s id=%s reason=%s",
        request.task, request.id, reason)
```

10.4 Alert Thresholds Summary

Metric	Warning	Critical
Queue depth (any queue)	> 100	> 500
Task failure rate (5 min)	> 1%	> 5%
Worker count	< 2 workers	0 workers
Task execution time (p99)	> 30s	> 120s
Redis memory (DB 1)	> 500 MB	> 800 MB
Beat scheduler last heartbeat	> 2 min	> 5 min

---

11. Production Worker Setup

11.1 systemd Service — Celery Worker

```
# /etc/systemd/system/bsecure-celery-worker.service
[Unit]
Description=b-secure Celery Worker
After=network.target redis.service postgresql.service
Wants=redis.service

[Service]
Type=forking
User=bsecure
Group=bsecure
WorkingDirectory=/opt/bsecure/backend
EnvironmentFile=/opt/bsecure/.env
ExecStart=/opt/bsecure/venv/bin/celery \
    -A celery_app multi start \
    worker-high worker-default worker-low \
```

```
    -Q:worker-high high_priority \
    -Q:worker-default default \
    -Q:worker-low low_priority \
    -c:worker-high 4 \
    -c:worker-default 8 \
    -c:worker-low 4 \
    --logfile=/var/log/bsecure/celery-%n.log \
    --pidfile=/var/run/bsecure/celery-%n.pid \
    --loglevel=info \
    --max-tasks-per-child=500
ExecStop=/opt/bsecure/venv/bin/celery \
    -A celery_app multi stopwait \
    worker-high worker-default worker-low \
    --pidfile=/var/run/bsecure/celery-%n.pid
ExecReload=/opt/bsecure/venv/bin/celery \
    -A celery_app multi restart \
    worker-high worker-default worker-low \
    --pidfile=/var/run/bsecure/celery-%n.pid \
    --logfile=/var/log/bsecure/celery-%n.log
Restart=on-failure
RestartSec=10s
StandardOutput=syslog
StandardError=syslog
SyslogIdentifier=bsecure-celery

[Install]
WantedBy=multi-user.target
```

11.2 systemd Service — Celery Beat

```
# /etc/systemd/system/bsecure-celery-beat.service
[Unit]
Description=b-secure Celery Beat Scheduler
After=network.target redis.service postgresql.service bsecure-
        celery-worker.service
Wants=redis.service

[Service]
Type=simple
User=bsecure
Group=bsecure
WorkingDirectory=/opt/bsecure/backend
EnvironmentFile=/opt/bsecure/.env
ExecStart=/opt/bsecure/venv/bin/celery \
    -A celery_app beat \
    --scheduler=django_celery_beat.schedulers:DatabaseScheduler \
    --logfile=/var/log/bsecure/celery-beat.log \
    --pidfile=/var/run/bsecure/celery-beat.pid \
    --loglevel=info
Restart=on-failure
RestartSec=30s
```

```
[Install]
WantedBy=multi-user.target

# Enable and start
sudo systemctl daemon-reload
sudo systemctl enable bsecure-celery-worker bsecure-celery-beat
sudo systemctl start  bsecure-celery-worker bsecure-celery-beat
sudo systemctl status bsecure-celery-worker
```

11.3 Supervisor Config (Alternative to systemd)

```
# /etc/supervisor/conf.d/bsecure_celery.conf
[program:bsecure-celery-high]
command=/opt/bsecure/venv/bin/celery -A celery_app worker
    --queues=high_priority --concurrency=4 --hostname=worker-
        high@%%h
    --loglevel=info --max-tasks-per-child=500
directory=/opt/bsecure/backend
user=bsecure
numprocs=1
autostart=true
autorestart=true
startsecs=10
stopwaitsecs=600
stdout_logfile=/var/log/bsecure/celery-high.log
stderr_logfile=/var/log/bsecure/celery-high-error.log

[program:bsecure-celery-default]
command=/opt/bsecure/venv/bin/celery -A celery_app worker
    --queues=default --concurrency=8 --hostname=worker-default@%%h
    --loglevel=info --max-tasks-per-child=500
directory=/opt/bsecure/backend
user=bsecure
numprocs=1
autostart=true
autorestart=true
startsecs=10
stopwaitsecs=600
stdout_logfile=/var/log/bsecure/celery-default.log
stderr_logfile=/var/log/bsecure/celery-default-error.log

[program:bsecure-celery-beat]
command=/opt/bsecure/venv/bin/celery -A celery_app beat
    --scheduler=django_celery_beat.schedulers:DatabaseScheduler
    --loglevel=info
directory=/opt/bsecure/backend
user=bsecure
numprocs=1
autostart=true
autorestart=true
startsecs=10
stopwaitsecs=60
```

```
stdout_logfile=/var/log/bsecure/celery-beat.log
stderr_logfile=/var/log/bsecure/celery-beat-error.log

[group:bsecure-celery]
programs=bsecure-celery-high,bsecure-celery-default,bsecure-
        celery-beat
```

11.4 ECS Task Definition — Celery Worker Container

```
{
  "family": "bsecure-celery-worker",
  "networkMode": "awsvpc",
  "requiresCompatibilities": ["FARGATE"],
  "cpu": "1024",
  "memory": "2048",
  "executionRoleArn": "arn:aws:iam::123456789:role/bsecure-ecs-
        task-execution",
  "taskRoleArn": "arn:aws:iam::123456789:role/bsecure-ecs-task",
  "containerDefinitions": [
    {
      "name": "celery-worker-default",
      "image": "123456789.dkr.ecr.ap-south-1.amazonaws.com/
        bsecure-backend:latest",
      "command": [
        "celery", "-A", "celery_app", "worker",
        "--queues=default",
        "--concurrency=8",
        "--loglevel=info",
        "--max-tasks-per-child=500"
      ],
      "environment": [
        {"name": "DJANGO_SETTINGS_MODULE", "value":
        "config.settings.production"}
      ],
      "secrets": [
        {"name": "DATABASE_URL",      "valueFrom":
        "arn:aws:ssm:ap-south-1:123456789:parameter/bsecure/prod/
        DATABASE_URL"},
        {"name": "REDIS_CELERY_URL",  "valueFrom":
        "arn:aws:ssm:ap-south-1:123456789:parameter/bsecure/prod/
        REDIS_CELERY_URL"},
        {"name": "SECRET_KEY",        "valueFrom":
        "arn:aws:ssm:ap-south-1:123456789:parameter/bsecure/prod/
        SECRET_KEY"}
      ],
      "logConfiguration": {
        "logDriver": "awslogs",
        "options": {
          "awslogs-group":         "/ecs/bsecure-celery-worker",
          "awslogs-region":        "ap-south-1",
          "awslogs-stream-prefix": "celery"
        }
      },
```

```
      "healthCheck": {
        "command": ["CMD-SHELL", "celery -A celery_app inspect
        ping -d celery@$HOSTNAME || exit 1"],
        "interval": 30,
        "timeout": 10,
        "retries": 3,
        "startPeriod": 60
      }
    }
  ]
}
```